

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Jane Open Access   

Dummy University Computing Laboratory, [optional: Address], Country

My second affiliation, Country

Joan R. Public¹  

Department of Informatics, Dummy College, [optional: Address], Country

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent convallis orci arcu, eu mollis dolor. Aliquam eleifend suscipit lacinia. Maecenas quam mi, porta ut lacinia sed, convallis ac dui. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse potenti.

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Jane Open Access:* (Optional) author-specific funding acknowledgements

Joan R. Public: [funding]

Acknowledgements I want to thank ...

1 Introduction

ELPI is a dialect of λ PROLOG (see [14, 15, 7, 12]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [13, 3, 4, 19, 8, 9]. Examples include the Hierarchy-Builder library-structuring tool [5] and Derive [17, 18, 11], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI gained a static analysis for determinacy [10] to help users tame backtracking. ROCQ users are familiar with functional programming but not necessarily with logic programming and uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checkers identifies predicates that behave like functions, i.e., predicates that commit to their first solution and leave no *choice points* (places where backtracking could resume).

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantic depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI's implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [16] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard PROLOG abstract machines [20, 1], but it makes reasoning about the scope of *cut* difficult. To address that limitation we introduce a tree-based semantics in which the branches pruned by *cut* are explicit and we prove the two semantics equivalent. Using the

¹ Optional footnote, e.g. to mark corresponding author



© Jane Open Access and Joan R. Public;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:13

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

```

Inductive P := IP of nat. Inductive D := ID of nat. Inductive V := IV of nat.

Inductive Tm :=
  | Tm_P of P
  | Tm_D of D
  | Tm_V of V
  | Tm_App of Tm & Tm.

Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.

```

■ Figure 1 Tm types

■ Figure 2 Unif type

tree-based semantics we then show that if every rule of a predicate passes the determinacy analysis, the call to a deterministic predicate does not leave any choice points.

2 Common code: the language

put unif and pro-
gram in variable
hides from types

Before going to the two semantics, we show the piece of data structure that are shared by the them. The smallest unit of code that we can use in the language is an atom. The atom inductive (see Type 1) is either a cut or a call. A call carries a term (see Figure 1). A term (Tm) is either a predicate, a datum, a variable or the binary application of two terms.

```

Inductive A := cut | call : Tm → A.
Record R := mkR { head : Tm; premises : list A }.
Record P := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V → Tm}.
Definition bc : Unif → P →  $\mathcal{F}_v$  → Tm →
  Σ →  $\mathcal{F}_v$  * seq (Σ * seq A) :=

```

A rule (see Type 2) has a head of type term and a list of premises, that are a list of atoms. A program (see Type 3) is made by its rules and the signatures of predicates, **sigT** is a mapping from predicates to their signatures, signatures represent the definition of type from the simply typed lambda calculus, i.e. it is either a base type or an arrow for term application. We decorate arrows to know the mode (input or output) of the application argument.

A substitution (see Type 4) is a mapping from variables to terms. It is the output of a successful query. The empty substitution is noted ϵ .

The backchain function (bc, see Type 5) takes: a unificator U ; a program P ; a set fv of free variables ($\mathcal{F}_v$); a query q ; and the substitution σ . A backchain is performed when the current atom is a **call**. In this case, it start by recursively *dereferencing* the variables in σ , i.e. replacing all the variables in the query by their term mapping in σ .

TODO: free variables and program freshness

Then it takes the head of the term, which should be rigid, i.e. a predicate p , and look for the signature s of p in the program. Then, for each rule r in P , it unifies the head of r with q . Unification is possible thanks to the unificator U , whose type is shown in Figure 2. U explains how to unify terms up to standard unification (for output terms) or matching (for input terms). The function *unif* (resp. *matching*) unifies (resp. matches) the two terms under σ , it return an extension of σ if the two terms can be unified (resp. matched), and None otherwise. The result of a backchain is made by filtering the rules in P that can be used on the given query. In particular, for each rule filtered rules r , it returns a pair containing the extension of σ making the head of r and q unify and the list of premises of r .

```

f 1 2.   f 2 3.   r 0 0.   r 0 1.   r 0 2.
g X Z :- r X Y, f Y Z, !. % r1
g X Z :- f X Y, f Y Z.    % r2

```

Inductive tree :=
| KO | OK | TA of A
| Or of option tree & Σ & tree
| And of tree & seq A & tree.

■ **Figure 3** Small ELPI program example

■ **Figure 4** Tree inductive

2.1 The cut operator

ELPI is a dialect of λ PROLOG with the hard cut operator and, in the language we have shown before, *cut* is an atom. The semantics of the cut operator adopted in the ELPI language corresponds to the *hard cut* operator of standard SWI-PROLOG. This operator has two primary purposes. First, it eliminates all alternatives that are created either simultaneously with, or after, the introduction of the cut into the execution state.

The ELPI program we will use as an example in the following sections is shown in Figure 3.

To illustrate the high-level description of *cut*, consider the program P shown in Figure 3 and the query $q = g \ 0 \ R$. Both rules for g can be used on the query q . Backchaining q in P from the empty (ϵ) substitution returns the list $[(\sigma_1, [r \ X \ Y, f \ Y \ Z, !]), (\sigma_2, [f \ X \ Y, f \ Y \ Z])]$ with σ_1 and σ_2 both equal to $\{X \mapsto 0, Z \mapsto R\}$. The two elements of the list are generated respectively from the rules r_1 and r_2 .

The next step of the interpretation corresponds to the backchaining of $r \ X \ Y$ from σ_1 . This returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [])]$ with $\sigma_3 := \sigma_1 + \{Y \mapsto 0\}$, $\sigma_4 := \sigma_1 + \{Y \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{Y \mapsto 2\}$. The three lists of premises are empty since all rules for r have no premises.

Backchaining $f \ Y \ Z$ in σ_3 gives no solution, therefore, by backtracking, we backchain $f \ Y \ Z$ in σ_4 . This gives a solution, namely $\sigma_6 := \sigma_4 + \{R \mapsto 2\}$.

We now reach the cut operator and can see its double side behavior. In the current execution state, we have still two unexplored alternatives: we can backtrack on the third solution for r that assign Y to 2 and on the rules r_2 . We can note, however, the first alternative have been generated after the cut introduction, and the second alternative have been generated at the same moment of the cut. Therefore both are cut away leaving no choice point. The final complete solution is: $\{X \mapsto 0, Z \mapsto R, Y \mapsto 1, R \mapsto 2\}$, where X , Y and Z can be ignored since they are the result internal unification.

3 Semantics intro

We propose two operational semantics for a logic program with cut. The two semantics are based on different syntaxes, the first syntax (called *tree*) exploits a tree-like structure and is ideal both to have a graphical view of its evolution while the tree is being interpreted and to prove lemmas over it. The second syntax, called *elpi*, is the ELPI's syntax and has the advantage of reducing the computational cost of cutting and backtracking alternatives by using shared pointers. We aim to prove the equivalence of the two semantics together with some interesting lemmas of the cut behavior.

4 Tree semantics

The tree state The tree inductive, shown in Figure 4. We distinguish 5 main cases: KO , OK , and are special meta-symbols representing, respectively, the basic failed and a successful

se metti $r1 = g \ A$
 $B :- f \ A \ B$. allora
 $g \ e \ f$ sono fun-
 zioni, e puoi spie-
 gare anche l'idea
 del detcheck qui

```

Fixpoint get_end s A :  $\Sigma$  * tree :=
  match A with
  | TA _ | KO | OK  $\Rightarrow$  (s, A)
  | Or None s1 B  $\Rightarrow$  get_end s1 B
  | Or (Some A) _ _  $\Rightarrow$  get_end s A
  | And A _ B  $\Rightarrow$ 
    let (s', pA) := get_end s A in
    if pA == OK then get_end s' B
    else (s', pA)
  end.

Definition get_subst s A := (get_end s A).1.
Definition path_end A := (get_end  $\epsilon$  A).2.
Definition success A := path_end A == OK.
Definition failed A := path_end A == KO.
Definition path_atom A :=
  if path_end A is TA _ then true
  else false.

```

(a) Definition of *get_end*(b) Functions from *get_end*

■ **Figure 5** *get_end* and derives functions

110 trees. These symbols are considered meta because they are internal intermediate symbols
 111 used to give structure to the tree.

112 The *TA* constructor (acronym for tree-atom) is the constructor of atoms in the tree. They
 113 are either the cut operator or a call, in these cases, the interpreter should evaluate the atom
 114 by either cutting the subtrees in the scope of the cut or backchaining the call in the program.

115 The two recursive cases of a tree are the *Or* and *And* nodes. The *Or* node $A \vee B_\sigma$ denotes
 116 a disjunction between two trees *A* and *B*. The first branch is optional, if absent it represents
 117 a dead tree, i.e. a tree that has been entirely explored. The second branch is annotated
 118 with a suspended substitution σ so that, upon backtracking to *B*, σ is used as the initial
 119 substitution for the execution of *B*.

120 The *And* node $A \wedge_{B_0} B$ represents a conjunction of two trees *A* and *B*. We call B_0 the
 121 reset point for *B*; it is used to restore the state of *B* to its initial form if a backtracking
 122 operation occurs on *A*. An example of this behavior is shown in Section 4.2

123 A graphical representation of a tree is shown in Figure 6a. To make the graph more
 124 compact, the *And* and *Or* nodes are n-ary rather than binary, and the children of these
 125 nodes associate to the right. The *KO* node acts as the neutral elements in the *Or* list, while
 126 *OK* is the neutral element of the *And* list.

127 *The tree interpreter* The interpretation of a tree is performed by two main routines: *step*
 128 and *next_alt* that traverse the tree depth-first, left-to-right. Then, then *runT* inductive
 129 makes the transitive closure of *step* and *next_alt*: it iterates the calls to its these
 130 auxiliary functions as much as needed. In Types 7–9 we give the types contrats of these
 131 symbols where $\mathcal{F}_v$ is a set of variable names.

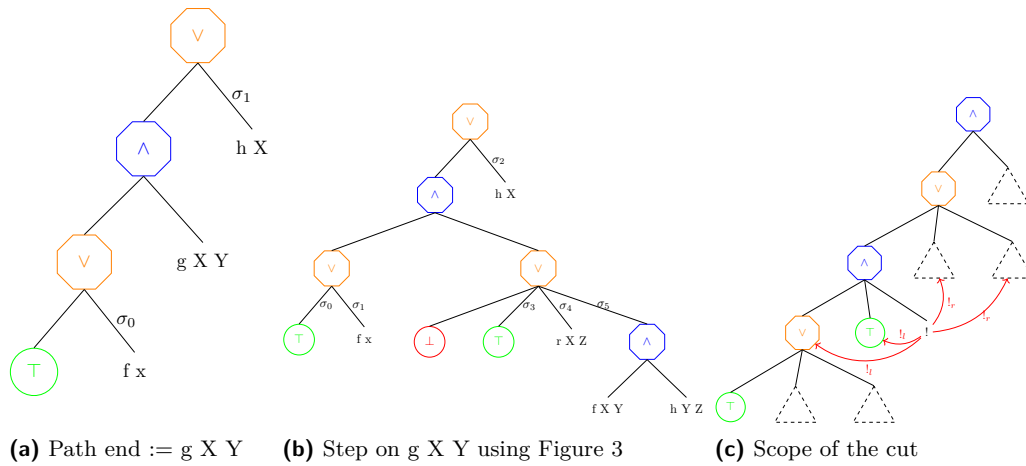
132 **Inductive** tag := Expanded | CutBrothers | Failed | Success. (6)

133 **Definition** step : $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree}) :=$ (7)

134 **Definition** next_alt : $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree} :=$ (8)

135 **Inductive** runT (u:Unif) (p : $\mathbb{P}$): $\mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow$
 $\Sigma \rightarrow \text{option tree} \rightarrow \text{Prop} :=$ (9)

136 The tree interpreter, as in prolog, explores the tree in DFS strategy, to discover the
 137 substitution and the leaf of the tree that should be interpreted. These two pieces of
 138 information can be retrieved for a substitution *s* and a tree *A* thanks to the *get_end* routine,
 139 shown in Figure 5a If the tree is already a leaf, *get_end* returns its inputs. Otherwise, if the
 140 tree is a disjunction, the path continues on the left subtree, if it exists, otherwise *get_end*
 141 contues on the ths using the substitution stored in the *Or* node. In the case of a conjunction,



■ **Figure 6** Some tree representations

142 if *get_end* of the lhs returns the tree *OK*, then the *get_end* continues in the rhs, otherwise
 143 we return the result of the lhs. We derive some useful definition from *get_end*. They are
 144 listed in Figure 5b.

145 The *path_end* of the tree in Figure 6a is g X Y.

146 4.1 The *step* procedure

147 The *step* procedure takes as input a program, a set of variables, a substitution, and a tree,
 148 and returns an updated set of variables, a *tag*, and an updated tree.

149 The set of variables *fv* given in input to *step* are used and updated when a backchaining
 150 operation takes place. A call to *step* produces a good result if *fv* contains all the variables in
 151 the tree. This guarantees that any call to *bc* returns a list of atoms whose variables
 152 are disjoint from the variables of the current working tree.

153 The *tag* (see Type 6) indicates the kind of an internal tree step: **CutBrothers** denotes the
 154 interpretation of a superficial cut, i.e., a cut whose parent nodes are all *And*-nodes. **Expanded**
 155 denotes the interpretation of non-superficial cuts or predicate calls. **Failure** and **Success** are
 156 returned for, respectively, *failed* and *success* trees.

157 The step procedure is intended to interpretate atoms, that is, it transforms the tree iff its
 158 *path_end* is an atom, otherwise, it returns the identity.

159 **Lemma** *succ_step_iff* u p fv s A: success A $\leftrightarrow$ step u p fv s A = (fv, Success, A). (1)

160 **Lemma** *fail_step_iff* u p fv s A: failed A $\leftrightarrow$ step u p fv s A = (fv, Failed, A). (2)

161 The interpretation of a call to a term *c* starts by calling the *bc* function on *c*. The output
 162 list *l* is taken to represent build the new subtree. If *l* is empty then *KO* tree is returned,
 163 otherwise the subtree is a right-skewed tree made of *n* inner *Or* nodes, where *n* is the length
 164 of *l*. The root has *KO* as left child. The lhs of the other nodes is a right-skewed tree of *And*
 165 nodes. The *And* nodes are again a right-skewed tree containing premises of the selected rule.

166 A step in the tree of Figure 6a makes a backchain operation over the query g X Y and, in
 167 the program defined in Figure 3, the new tree would be the one in Figure 6b. We have put a
 168 red border around the new generated subtree. It is a disjunction of four subtrees: the first
 169 node is the *KO* node (by default), the second is *OK*, since *r1* has no premises, the third and
 170 the fourth contains the premises of respectively *r2* and *r3*.

dire dei reset
point

dire che le
sostituzioni del
backchain sono
importanti e

171 The interpretation of a cut has three main impacts: at first the cut is replaced by the
 172 OK node, then some special subtrees, in the scope of the *Cut*, are cut away: in particular we
 173 need to soft-kill the left-siblings of the *Cut* and hard-kill the right-uncles of the the *Cut*.

174 ► **Definition 1** (Left-siblings (resp. right-sibling)). Given a node A , the left-siblings (resp.
 175 right-sibling) of A are the list of subtrees sharing the same parent of A and that appear on
 176 its left (resp. right).

177 ► **Definition 2** (Right-uncles). Given a node A , the right-uncles of A are the list of right-sibling
 178 of the father of A .

179 ► **Definition 3** (Hard-kill, $!_r$). Given a tree t , hard-kill replaces the given subtree with the
 180 KO node

181 ► **Definition 4** (Soft-kill, $!_l$). Given a successfull tree t , soft-kill replaces with KO all subtrees
 182 that are not part of the path in t leading to the OK node.

183 An example of the impact of the cut is show in Figure 6c, the dashed triangles represent
 184 generic trees. The step routine interprets the cut since it is the node in its path-end: we pass
 185 through a and and all trees on the left of the cut are successful. In the example we have 4
 186 arrow tagged with the $!_l$ or $!_r$ symbols. The $!_l$ arrows go left and soft-kill the pointed subtree,
 187 it keeps OK nodes since they are part of the tree leading to the cut, and replaces the other
 188 subtrees with KO. The $!_r$ procedure replaces the nodes pointed by the arrows with KO.

189 4.2 The *next_alt* procedure

190 It is evident that the *step* alone is not sufficient to reproduce entirely the behavior of the
 191 full expected prolog interpreter. In particular, we need to backtrack on failures. Moreover,
 192 in case of success, we should return a tree cleaned of its success, this is essential to, non
 193 deterministically, find the next solution of a given query. By Lemmas 1 and 2, we know that
 194 *step* returns the identity on successful and failed trees. In order to continue the computation
 195 on these particular trees, we need the *next_alt* procedure aiming to expecially work with
 196 failed and successful trees: and its implementation in Figure 7.

197 The *next_alt* procedure takes a boolean and a tree, clean it from failures or success and
 198 returns a new tree if this tree still contains a non explored path. The idea behind *next_alt* is
 199 to clean recursively every subtree in DFS order if its *path_end* is a failure. Moreover, if the
 200 boolean passed to *next_alt* is true, then it erases the first successful path in the tree.

201 The base cases of *next_alt* are immediate. The *Or* case is rather intuitive: if the lhs
 202 of the *Or* does not exist we look for the *next_alt* in the rhs. Otherwise, we look for the
 203 *next_alt* in the lhs, if this *next_alt* does not exists, we look for the *next_alt* in the rhs.

204 We want to spend few words about the *And* case, since the reset point $B0$ for B plays an
 205 important role. The *next_alt* in an *And* tree should consider two cases: if the lhs succeeds,
 206 then the *next_alt* should be retrived in the rhs. If this alternative does not exists it means
 207 that the rhs has entirely been explored. We need to erase the success in the lhs and try to
 208 find if a non-explored alternative exists. If so, we return a new tree with the new lhs and the
 209 rhs is built from the reset point. *big_and* is a trivial function build a right-skewed tree of
 210 and nodes where the leaves are the atoms written in the reset point. We need to reuse the
 211 reset point since, the step procedure in *And* trees evaluates the rhs of a *And* tree if the lhs
 212 succeeds. This evaluation is dependent on the substitution in the lhs tree. Therefore, if we
 213 need to backtrack in the lhs, we need to reset the rhs.

214 Some interesting property of *next_alt* are shown below and allow to see how *next_alt*
 215 complements *step*.

```

Definition next_alt :  $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree} :=
  \text{fix next\_alt } b \ A :=
  \text{match } A \text{ with}
  | KO \Rightarrow \text{None}
  | OK \Rightarrow \text{if } b \text{ then None else Some OK}
  | TA _ \Rightarrow \text{Some } A
  | And A BO B \Rightarrow
    \text{let build\_BO } A := \text{And } A \text{ BO } (\text{big\_and } BO) \text{ in}
    \text{if success } A \text{ then}
      \text{match next\_alt } b \ B \text{ with}
      | None \Rightarrow \text{omap build\_BO } (\text{next\_alt true } A)
      | Some B' \Rightarrow \text{Some } (\text{And } A \text{ BO } B')
    \text{end}
    \text{else if failed } A \text{ then omap build\_BO } (\text{next\_alt false } A)
    \text{else Some } (\text{And } A \text{ BO } B)
  | Or None sB B \Rightarrow \text{omap } (\text{fun } x \Rightarrow \text{Or None sB } x) (\text{next\_alt } b \ B)
  | Or (Some A) sB B \Rightarrow
    \text{match next\_alt } b \ A \text{ with}
    | None \Rightarrow \text{omap } (\text{fun } x \Rightarrow \text{Or None sB } x) (\text{next\_alt false } B)
    | Some A' \Rightarrow \text{Some } (\text{Or } (\text{Some } A') \text{ sB } B)
    \text{end}
  \text{end.}$ 
```

■ **Figure 7** *next_alt* implementation

216 **Lemma** *path_atom_next_alt_id* $b \ A$: $\text{path_atom } A \rightarrow \text{next_alt } b \ A = \text{Some } A$. (3)

217 **Lemma** *next_alt_failedF* $b \ A \ A'$: $\text{next_alt } b \ A = \text{Some } A' \rightarrow \text{failed } A' = \text{false}$. (4)

218 For example, in Figure 6b the step procedure has created a failed tree: its path-end ends
 219 in *KO*. The expected behavior of *next_alt* is to take this *KO* node and make it a This
 220 allows *step* to continue the exploration of the tree. In particular, the path-end of this new
 221 tree ends in *OK*. The step leaves the tree unchanged producing the new substitution. This
 222 solution however is not unique, we should be able to backtrack on this successful tree. To do
 223 so we can call *next_alt* and it will deadify the *OK* node allowing *step* to proceed on $r \ X \ Z$.

subst taken from
the or

224 4.3 The *runT* inductive

225 TODO: start

226 DIRE CHE RUN TERMINA

227 In our implementation, *runT* gives the guarantee that the program interpretation terminates,
 228 since it always return a substitution, but we can easily make it also accept diverging program
 229 by making *runT* return an optional substitution

230 TODO: end

231 The inductive procedure *runT* is modeled as a function: it takes as input a program, a
 232 set of free variables, an initial substitution σ_0 , and a tree t_0 , and returns a substitution σ_1
 233 together with an optional updated tree t_1 . The substitution σ_1 represents the most-general
 234 unificator that makes the execution of the tree t_0 succeed starting from the initial substitution
 235 σ_0 , σ_1 is an extension of σ_0 . The tree t_1 is the updated tree containing the alternatives that
 236 have not yet been explored. If the tree contains no solution, then *None* is returned.

237 The procedure *runT* is based on three main derivation rules, shown in Figure 8. If the
 238 *path_end* of the tree t is a success, the input substitution is returned and the input tree is

cleaned of its successful path. If the *path_end* of the tree is an atom, then *step* is invoked to evaluate this atom, and *runT* is recursively called on the new tree. Finally, if the *path_end* of the tree is a failure, *next_alt* is called to clear the failed path; if the resulting cleaned tree exists, *runT* is recursively called on it.

$$\begin{array}{c}
\frac{\text{success } A \quad \text{get_subst } s_1 \ A = s_2 \quad (\text{next_alt true } A) = B}{\text{runT fv } s_1 \ A \ s_2 \ B} \text{ RUN_DONE} \\
\\
\frac{\text{path_atom } A \quad \text{step u p fv0 } s_1 \ A = (\text{fv1}, \text{st}, B) \quad \text{runT fv1 } s_1 \ B \ s_2 \ r}{\text{runT fv0 } s_1 \ A \ s_2 \ r} \text{ RUN_STEP} \\
\\
\frac{\text{failed } A \quad \text{next_alt false } A = \text{Some } B \quad \text{runT fv0 } s_1 \ B \ s_2 \ r}{\text{runT fv0 } s_1 \ A \ s_2 \ r} \text{ RUN_FAIL}
\end{array}$$

■ **Figure 8** Rule system for *runT*

242

243 **5 Elpi semantics**

244 We now want to introduce the elpi semantics. The interpreter we show reflects the interpreter
 245 of the ELPI language and is an operational semantics close to the one picked by Pusch in
 246 [16], in turn closely related to the one given by Debray and Mishra in [6, Section 4.3]. Pusch
 247 mechanized the semantics in Isabelle/HOL together with some optimizations that are present
 248 in the Warren Abstract Machine [20, 1].

249 The inductive representing a state of the ELPI language is shown below.

```

Inductive alts := no_alt | more_alt of (Σ * goals) & alts
with goals := no_goals | more_goals of (A * alts) & goals .

```

250 An elpi state is an enhanced two-dimension list. The outermost list represents the list
 251 of alternatives in disjunction accompagnate with the substitution that should be used to
 252 for their interpretation. The innermost list is a list of atom, representing a list of goals in
 253 conjunctions. These goals are decorated with a pointer to an elpi state, and are used to keep
 254 trace of the alternatives that should be kept when a cut is interpreted. We call these, special,
 255 alternatives the cut-to alternatives.

256 The idea of the ELPI interpreter is to receive a list of alternatives. The first alternative
 257 consists of a list of goals. Four cases must be taken into account; they are shown in
 258 Figure 9. In order to simplify goal retrieval, we split the head of the alternatives from the
 259 tail, so that it can be immediately matched in the inductive definition. Note that an empty
 260 list of alternatives represents, by definition, a failing elpi state. If the goal list is empty
 261 (STOPE), then we have, by definition, a success, and the input solution together with the
 262 list of alternatives is returned. If the goal list starts with a cut (CUTE), then the current
 263 alternatives are erased in favour of the cut alternatives, and a recursive call is made on the
 264 remaining goal list.

265 Finally, we must consider the case in which the goal list starts with a call. The call
 266 can either fail (FAILE) or succeed (CALLE). We distinguish the two cases by looking if the
 267 backchaining operation returns zero or more rules. We have wrapped this task in the *stepE*
 268 procedure, which also updates the goal and cut-alternative list. The fail case, is relatively
 269 easy: the first goal does not succeed, we need to take the head of the alternatives, and make
 270 it the new list of goals to be explored.

Definition $\text{stepE } fv \ t \ s \ a \ gl :=$
 $\text{let } (fv', rs) := \text{bc_up } fv \ t \ s \ \text{in}$
 $\text{let } rs_ca := \text{save_alts } a \ gl \ (r2a \ rs) \ \text{in}$
 $(fv', rs_ca).$

Inductive $\text{nur} : \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{goals} \rightarrow \text{alts} \rightarrow \Sigma \rightarrow \text{alts} \rightarrow \text{Prop} :=$ (10)

$$\frac{}{\text{nur } fv \ s \ [::] \ a \ s \ a} \text{STOPE}$$

$$\frac{\text{nur } fv \ s \ gl \ ca \ s_1 \ r}{\text{nur } fv \ s \ [:: (\text{cut}, ca) \ \& \ gl] \ a \ s_1 \ r} \text{CUTE}$$

$$\frac{\text{stepE } fv \ t \ s \ al \ gl = (fv', [:: b \ \& \ bs]) \quad \text{nur } fv' \ b.1 \ b.2 \ (bs++al) \ s_1 \ r}{\text{nur } fv \ s \ [:: (\text{call } t, ca) \ \& \ gl] \ al \ s_1 \ r} \text{CALLE}$$

$$\frac{\text{stepE } fv \ t \ s \ al \ gl = (fv', [::]) \quad \text{nur } fv' \ s_1 \ a \ al \ s_2 \ r}{\text{nur } fv \ s \ [:: (\text{call } t, ca) \ \& \ gl] \ [:: (s_1, a) \ \& \ al] \ s_2 \ r} \text{FAILE}$$

■ **Figure 9** Rule system for *nur*

271 The case in which backchaining produces a non empty list, the *save_alts* routine is in
 272 cahrg of: taking the list of premises and add to each atom the the list of alternatives *a* as
 273 their new cut-alternatives, then it append the list of goals *gl* to each of these new lists.

274 6 Semantic equivalence

275 The equivalence between the two semantics is possible under two conditions: we need to
 276 work with “valid tree”, i.e. all of those trees that can be generated from a call. Secondly,
 277 we need to translate trees into elpi states. In the next to subsection we propose the two
 278 functions *valid_state* and *t2l*.

279 6.1 Valid trees

280 The inductive tree allows one to generate a large number of trees, some of which are not
 281 valid, in the sense that they cannot be produced starting from a given query. The class of
 282 valid trees is characterized by the function shown in Figure 10a.

283 While all base cases of tree are considered valid, we need to analyze carefully the cases
 284 for the *Or* and *And* constructors.

285 For the *Or* constructor, we distinguish two cases depending on whether the left-hand
 286 side (lhs) exists. If it does not exist, then the right-hand side (rhs) must be a valid tree.
 287 Otherwise, the lhs must itself be a valid tree, and the rhs is either the *KO* tree, since it may
 288 have been removed by the evaluation of a superficial cut in the lhs, or it has not yet been
 289 explored. In the latter case, it is a **base_or** tree, namely the right-skewed tree formed by a
 290 disjunction of conjunctions that is generated after a succesful backchain to a call.

291 For the *And* constructor, the lhs is required to be a valid tree. The shape of the rhs
 292 depends on whether the lhs is a success. If the lhs is not successful, then the rhs has never
 293 been explored: the procedures *step* and *next_alt* modify the rhs only when the lhs succeeds.
 294 In this case, the lhs must be the right-skewed tree containing conjunctions of premises atoms,

```

Fixpoint t2l A s0 bt :=
  match A with
  | OK           ⇒ [:: (s0, [::])]
  | KO           ⇒ [::]
  | TA a         ⇒ [:: (s0, [:: (a, [::]) ])]
  | Or None s1 B ⇒ add_ca_deep bt (t2l B s1 [::])
  | Or (Some A) s1 B ⇒
    let lB := t2l B s1 [::] in
    let lA := t2l A s0 lB in
    add_ca_deep bt (lA ++ lB)
  | And A B0 B ⇒
    let lA := t2l A s0 bt in
    let lB0 := a2g B0 in
    let lA := add_deep bt lB0 lA in
    if lA is [:: (s0, gs) & al] then
      let al := map (catr lB0) al in
      let lB := t2l B s0 (al ++ bt) in
      map (catl gs) lB ++ al
    else [::]
end.

Fixpoint valid_tree A :=
  match A with
  | TA _ | OK | KO ⇒ true
  | Or None _ B ⇒ valid_tree B
  | Or (Some A) _ B ⇒ valid_tree A &&
    ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
    if success A then valid_tree B
    else B == big_and B0
end.

```

(a) Valid tree

(b) Tree to list

■ Figure 10 Valid tree and Tree to list

the reset point B_0 ensure what shape the rhs should have. If the lhs is a success tree, then the rhs can be modified by *step* and *next_alt*, therefore it must be a valid tree.

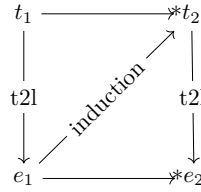
6.2 From trees to lists

The translation of a tree to a list is shown in Figure 10b. It takes the tree to be translated, a substitution (called s), a list of alternatives, called bt . The substitution s tells what is the substitution of the alternative we are building and is updated when going to the rhs of the *Or* constructor. The bt list represents the future alternatives list that have the same depth of the current tree root. They are useful to know if they should be added to the current goal as cut alternatives.

An *OK* node represent a success in a tree, then the translation of this tree returns a singleton list with the couple $(s, [::])$: there is one alternative with 0 goals, i.e. a success in elpi by *STOPE*. The *KO* node represent a failure, and, therefore 0 alternatives. An atom is translated into a singleton with the atom as first goal and the empty cut-alternative list: we are not adding bt since they are the alternatives for the right-sibling, not the right-uncles, this means that if we have a is a cut, then it erases the alternatives bt .

In a disjunction, we are scendendo di un livello the distance from our backtracking list. This means that bt becomes the list of right-uncles of the two branches of the *Or* constructor. The compilation of the rhs is done independently of if the lhs exists: we transform the rhs into an elpi state and passing the empty list of alternatives, since the rhs as no right-siblings. Then, thanks to *add_ca_deep*, we recursively add bt to every cut-alternative appearing in the translated goals. If the lhs exists, we translate it and pass the translation of the rhs as the list of right-siblings. Then we prepend the translated lhs to the translated rhs and add bt with *add_ca_deep*.

We compile the *And* case by translating its lhs to the list lA and reset point to the list



■ **Figure 11** Induction scheme for Lemma 6

319 *lB0*. Then, thanks to `add_deep`, we recursively append *lB0* to the alternatives born in *lA*,
 320 i.e. we leave unchanged the pointers to *bt*, if any, in *lA*. We finally need to treat cases
 321 whenever the list *lA* is empty or not. In the former case the empty list is returned, otherwise,
 322 we have a list $[(s_0, gs) \& al]$. By the semantics of the *And* in the *runT* interpreter, the rhs
 323 of the *And* represent the sequent of goals to be executed after the first alternative in the
 324 lhs, it corresponds to the list of goals *gs*. The reset point *B0* is to be appended to the next
 325 alternatives in the lhs, that is the queue of *lA*, that is *al*.

alternative in
tree? rephrase

326 We note therefore that the compilation of *lB* is done by passing the alternatives *al* + *bt*
 327 since the alternatives in *al* are to be...

328 6.3 Equivalence theorems

329 We have now all the ingredients that ...

Lemma *tree_to_elpi*: $\forall u p s_0 t s_2 t',$
`let` *fv* := `vars_tree t` | `vars_sigma s_0` **in**
`valid_tree t` $\rightarrow$
`runT u p fv s_0 t s_2 t'` $\rightarrow$
 330 $\exists s_1 g a,$
`let` *na* := `(t2l (odflt K0 t')) s_0 [::]` **in**
`t2l t s_0 [::] = (s_1, g) :: a /\`
`nur u p fv s_1 g a s_2 na.` (5)

Lemma *elpi_to_tree*: $\forall u p fv s_1 g s_2 a na,$
`nur u p fv s_1 g a s_2 na` $\rightarrow$
 331 $\forall s_0 t, \text{valid_tree } t \rightarrow \text{t2l } t s_0 [::] = (s_1, g) :: a \rightarrow$ (6)
 $\exists t', \text{runT } u p fv s_0 t s_2 t' /\ \text{t2l } (\text{odflt } K0 t') s_0 [::] = na.$

332 The proof of Lemma 6 is based on the idea explained in [2, Section 3.3]. An ideal
 333 statement for this lemma would be: given a function *l2t* transforming an elpi state to a
 334 tree, we would have have that the the execution of an elpi state *e* is the same as executing
 335 *runT* on the tree resulting from *l2t(e)*. However, it is difficult to retrieve the strucutre of
 336 an elpi state and create a tree from it. This is because, in an elpi state, we have no clear
 337 information about the scope of an atom inside the list and, therefore, no evident clue about
 338 where this atom should be place in the tree.

339 Our theorem states that, starting from a valid tree *t* which translates to a list of alternatives
 340 $(\sigma_1, g) :: a$. If we run in elpi the list of alternatives, then the execution of the tree *t* returns the
 341 same result as the execution in elpi. The proof is performed by induction on the derivations
 342 of the elpi execution. We have 4 derivations.

343 We have 4 case to analyse:

7 Case study: determinacy analysis

we mechanize the first order part of xxx.
 snippet det, main thm, invariant det tree (valid tree prev section?)
 proof inducition on exec, step/next alt preserving invariant proved by induction on the tree.
 with list semantics cut and next alt requires to express a link between the ca or next alts and the current goal, which is non trivial without an intermediate data strature like the tree

8 Related work

prolog semantics, King lost
 yves for the proof technique

9 Conclusion

References

- 1 Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- 4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
- 5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPICs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.FSCD.2020.34>, doi:10.4230/LIPICs.FSCD.2020.34.
- 6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.
- 7 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7_32.
- 8 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 9 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
- 10 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In *Practical Aspects of Declarative Languages: 28th International Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95, Berlin, Heidelberg, 2026. Springer-Verlag. doi:10.1007/978-3-032-15981-6_5.

- 388 11 Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality
389 tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing
390 Machinery, 2023. doi:10.1145/3573105.3575683.
- 391 12 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory
392 in higher order constraint logic programming. In *Mathematical Structures in Computer
393 Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/
394 S0960129518000427.
- 395 13 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least
396 Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors,
397 *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leib-
398 nitz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany,
399 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: [https://drops.dagstuhl.
400 de/entities/document/10.4230/LIPIcs.ITP.2025.27](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27), doi:10.4230/LIPIcs.ITP.2025.27.
- 401 14 Dale Miller. A logic programming language with lambda-abstraction, function variables, and
402 simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 403 15 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge
404 University Press, 2012.
- 405 16 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris
406 Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors,
407 *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer
408 Berlin Heidelberg.
- 409 17 Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog
410 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
411 2018. URL: <https://inria.hal.science/hal-01637063>.
- 412 18 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
413 *LIPIcs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
414 doi:10.4230/LIPIcs.CVIT.2016.23.
- 415 19 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
416 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 417 20 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
418 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
419 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/
420 2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).